ModBus

- Diseñado en 1979 por Modicon, Actualmente Schneider Automation

  - Público
  - Fácil de implementar
  - Maneja bloques de datos sin restricciones. La transmisión no está comprometida a ningún tipo de datos.

ModBus

- Es un protocolo de comunicación industrial, que podemos encontrar en fabricas y edificios inteligentes

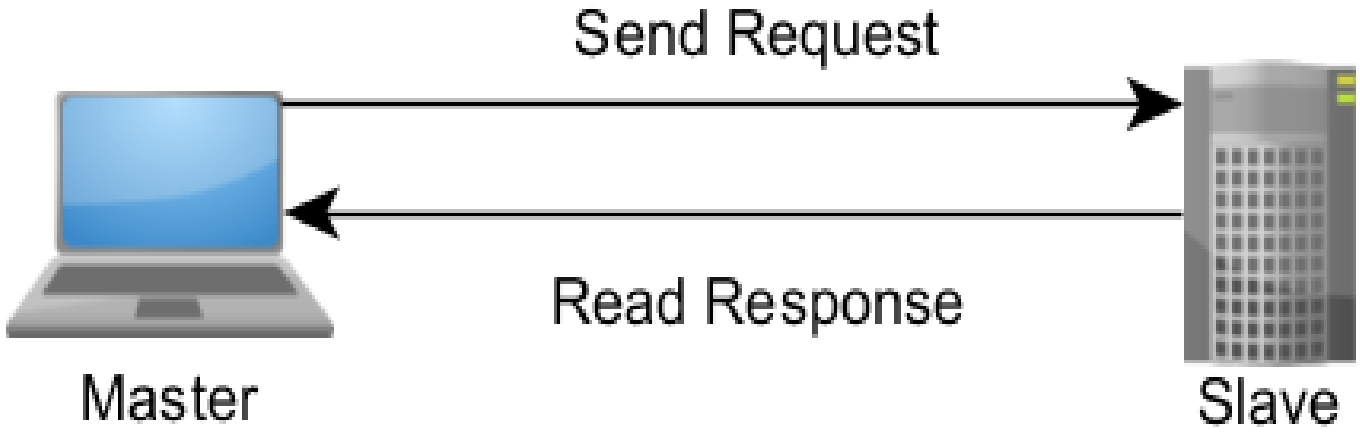
- Es el protocolo más usado y está integrado en todo tipo de industrias, edificios, y en todo tipo de dispositivos.

ModBus

- Se utiliza en una configuración ***Maestro-Esclavo***
  - un dispositivo debe iniciar una solicitud y luego esperar una respuesta.
  - El dispositivo de inicio (el maestro) es responsable de iniciar cada interacción

ModBus

- Por lo general, el maestro es una interfaz humano-máquina (HMI) o sistema SCADA (Supervisión, Control y Adquisición de Datos) y el esclavo es un sensor, controlador lógico programable (PLC) o controlador de automatización programable (PAC)



Capas de ModBus

- Inicialmente diseñado para comunicaciones serie

  → No se puede dividir en capas

- Con el tiempo, diferentes unidades de datos de aplicación fueron introducidas ya sea para cambiar el formato del paquete utilizado a través de serial o para permitir el uso de redes TCP/IP y UDP.

Capas de ModBus

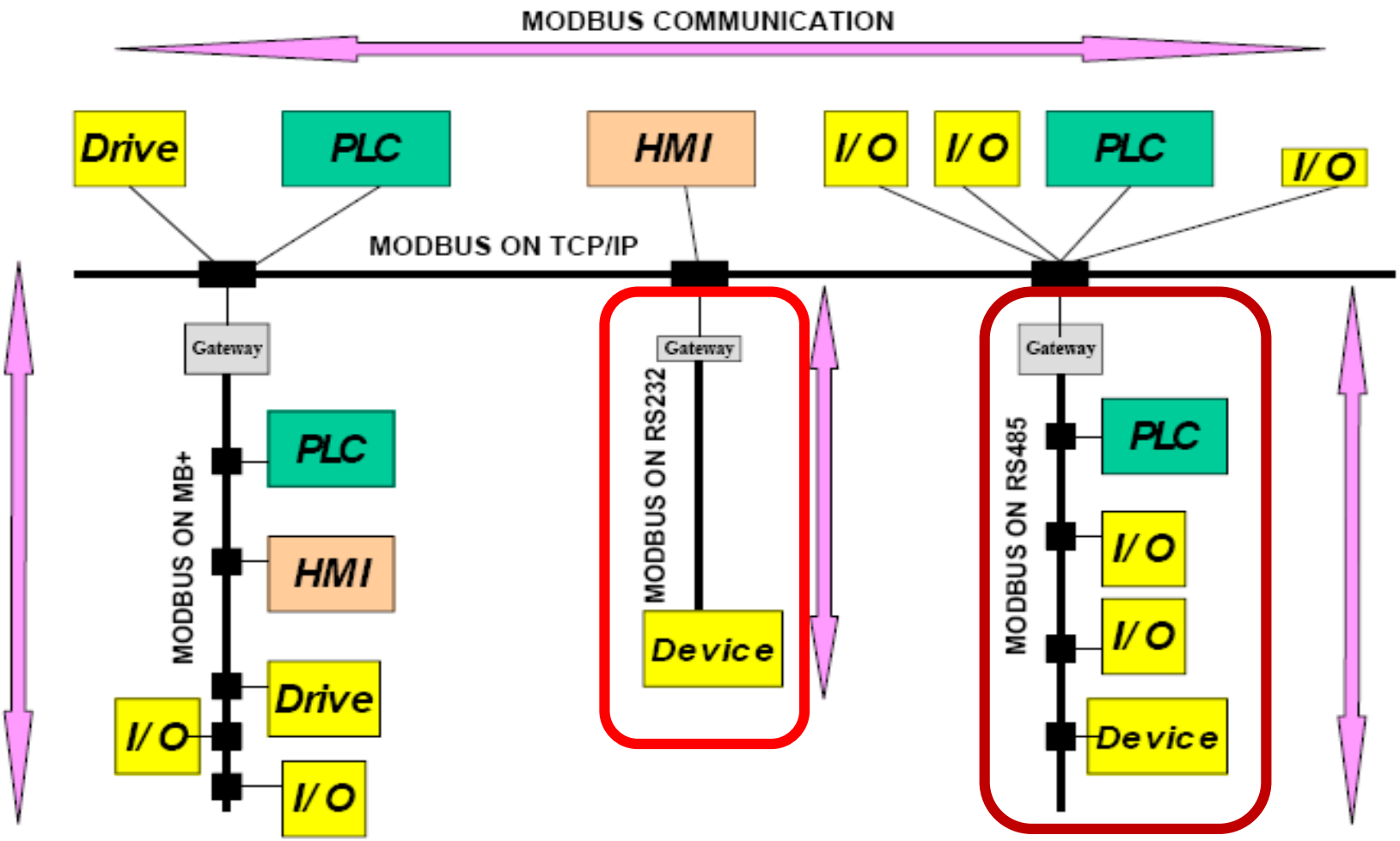
- Esto llevó a una separación del protocolo principal, el cual define la **unidad de datos de protocolo** (PDU) y la capa de red, que define la **unidad de datos de aplicación** (ADU).

  - PDU → Usadas para el intercambio de datos

    Código de función + Datos

  - ADU = Dirección + PDU + Comprobación de errores,

ModBus (Arquitectura de RED)



Unidad de Datos de Protocolo

- El código que la maneja consiste en el núcleo de la Especificación del Protocolo de Aplicación Modbus

http://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b.pdf

- El formato de Modbus PDU está definido como un código de función seguido por un conjunto de datos asociado.

ModBus – Trama

- Versiones del protocolo ModBus Serie

✓ ModBus RTU → Utiliza datos binarios y CRC

Silence >= 3.5 characters

✓ ModBus ASCII → más simple y fácil de leer, aunque menos eficiente, debido a que cada byte necesita dos bytes ASCII

Unidad de Datos de Protocolo

- Los datos disponibles por medio de Modbus son almacenados, en general, en uno de los cuatro bancos de datos o rangos de dirección:

  - bobinas (Salidas Digitales o Discretas)
  - entradas discretas
  - registros de retención (Salida)
  - registros de entrada

Unidad de Datos de Protocolo

Bloque de Memoria	Tipo de Datos	Acceso de Maestro	Acceso de Esclavo
Bobinas	Booleano	Lectura/Escritura	Lectura/Escritura
Entradas Discretas	Booleano	Solo Lectura	Lectura/Escritura
Registros de Retención	Palabra Sin Signo	Lectura/Escritura	Lectura/Escritura
Registros de Entrada	Palabra Sin Signo	Solo Lectura	Lectura/Escritura

- Los bloques son completamente conceptuales. Pueden existir como direcciones de memoria separadas en un sistema determinado, pero también pueden solaparse

Dirección de modelo de Datos

- El esquema de dirección se define completamente por el dispositivo esclavo y su interpretación de cada bloque de memoria es una parte importante del modelo de datos del dispositivo

- La especificación define que cada bloque contiene un espacio de dirección de un máximo de 65,536 (2^{16}) elementos (nuevo estándar)

- Cada elemento de datos está numerado de 1 a n, donde n tiene un valor máximo de 65,536 ➔ ***Bobina 1 → Address 0***

Rangos de Dirección de Datos

- ***No*** se requiere que los rangos completos permitidos por la especificación sean implementados por un determinado dispositivo

- Aunque la especificación define los *diferentes tipos de datos que existen en diferentes bloques* y *asigna un rango de dirección local para cada tipo*, esto no se traduce necesariamente en un esquema intuitivo de dirección. Para simplificar la discusión de ubicaciones del bloque de memoria, se introdujo un esquema de numeración, el cual añade prefijos a la dirección de los datos en cuestión

Rangos de Dirección de Datos

Bloque de Datos	Prefijo
Bobinas	0
Entradas Discretas	1
Registros de Entrada	3
Registros de Retención	4

- Existen bobinas con un prefijo 0. Esto significa que una referencia de 4001 podría referirse al registro de retención uno o bobina de 4001. Por esta razón, se recomienda que todas las nuevas implementaciones usen dirección de 6 dígitos con ceros a la izquierda y se especifique esto en la documentación. Por lo tanto, el registro de retención uno es referenciado como 400,001 y la bobina de 4001 es referenciada como 004,001

ModBus (Funcionamiento)

- Tiene una base sencilla
  - ✓ El **Master** Pregunta y los **Slaves** responden o actúan en función de lo que se les diga.

- Según el estándar, habrá un Master y hasta un máximo de 247 dispositivos Esclavos (Slaves).
  - ✓ La dirección del esclavo se representa con un solo byte.
  - ✓ Existen direcciones reservadas, como Broadcast.
  - ✓ Todos los esclavos deben tener un única dirección comprendida entre 1 y 247

ModBus RTU - Trama

Silencio	Dirección Esclavo	Función ModBus	Bytes de Datos	Checksum	Silencio
	1 Byte	1 Byte	N Bytes	2 Bytes	

Silencio >= 3.5 Caracteres

- A. Esclavo (Dirección Esclavo)
  - ✓Valor comprendido entre 1 – 247
  - ✓Cuando el master pregunta al slave este campo contiene la dirección del slave al que va dirigido. Cuando se trata de una trama de respuesta de un slave al master este campo contiene también la dirección del esclavo indicando quién es el que responde.

ModBus RTU - Trama

F. ModBus (Función ModBus)

✓Indica la acción que requiere el Master del Slave

✓*Varia si se trata de una trama Master → Slave o si por el contrario es Slave → Master.*

•*Si la trama del Maestro es correcta, la trama de respuesta tiene este byte con el mismo valor.*

•*Caso de error, el Esclavo responderá con la misma función que en la trama enviada por el Maestro, pero con la máscara 0x80 aplicada*

ModBus RTU – Trama Error

Mensaje Maestro

Dirección	Función	Reg. Com. (Alto)	Reg. Com. (Bajo)	Num. Reg. (Alto)	Num. Reg. (Bajo)	Control de Error
0A	01	04	A1	00	01	4F

Respuesta Esclavo

Dirección	Función	Código de Error	Control de Error
0A	81	02	73

ModBus RTU – Códigos Error

Código	Tipo Error	Significado
01	Función Ilegal	La función recibida no está permitida en el Esclavo
02	Dirección Ilegal	La dirección está fuera del rango permitido
03	Dato Ilegal	El dato contiene un valor no válido
04	Falla en el Dispositivo	El controlador no responde o ha ocurrido un error
05	Reconocimiento (ACK)	Se ha aceptado la función y se ha procesado
06	Ocupado	El mensaje ha sido recibido sin error, pero el dispositivo no puede procesarlo en este momento
07	Reconocimiento Negativo (NAK)	La función solicitada no puede realizarse en este momento

ModBus – Códigos de Clase 0

- El mínimo para un dispositivo Modbus, ya que dan a un maestro la habilidad de leer o escribir en el modelo de datos.

Código	Descripción
3	Leer Múltiples Registros
16	Escribir a Múltiples Registros

ModBus – Códigos de Clase 1

- Códigos necesarios para tener acceso a todos los tipos del modelo de datos.

Código	Descripción
1	Leer Bobinas
2	Leer Entradas Discretas
4	Leer Registros de Entrada
5	Escribir a Bobina Individual
6	Escribir a Registro Individual
7	Leer Estado de Excepción (únicamente serial)

ModBus – Códigos de Clase 2

- Funciones más especializadas e implementadas con menos frecuencia.

Código	Descripción
15	Escribir a Múltiples Bobinas
20	Leer Registro de Archivo
21	Escribir a Registro de Archivo
22	Escribir a Registro con Máscara
23	Leer/Escribir Múltiples Registros
24	Leer FIFO

Funciones de Lectura de Datos

Función	Código	Descripción
READ_COILS	01 (01 hex)	Lectura de Señales discretas de salida (Discrete Output Coils)
READ_INPUT_DISCRETE	02 (02 hex)	Lectura de señales discretas de entradas (Discrete Input Contacts)
READ_MULTIPLE_REGISTERS	03 (03 hex)	Lectura de registros analógicos (Analog Output Holding Registers)
READ_INPUT_REGISTERS	04 (04 hex)	Lectura de registros analógicos de entrada (Analog Input Registers)

Funciones de Escritura de Datos

Función	Código	Descripción
WRITE_COILS	05 (05 hex)	Escritura de una Señal discreta de salida (Simple Discrete Output Coils)
WRITE_SINGLE_REGISTER	06 (06 hex)	Escritura de un simple Analog Output Holding Register
WRITE_MULTIPLE_COILS	15 (0F hex)	Escritura de múltiples señales discretas de salida (Multiple Discrete Output Coils)
WRITE_MULTIPLE_REGISTERS	16 (10 hex)	Escritura Múltiple Analog Output Holding Registers

ModBus RTU - Trama

Datos (Bytes de Datos)

✓*Dependerá tanto en contenido como en longitud de la función que se indique.*

✓*También, si se trata de una trama **Master-Slave** o de respuesta **Slave-Master***

✓***La transferencia de datos está organizada en registros de 16-bits.***

✓*Hay extensiones a :*

•***32-Bit-Integer** y **32-Bit-Float** → Transmitido como 2 registros de 16-Bits consecutivos*

•***64-Bit-Integer** y **64-Bit-Float** → Transmitido como 4 registros de 16-Bits consecutivos*

Ejemplo: Leer estado de dispositivos digitales

Pregunta : Maestro → Esclavo						
Dirección	Función	Datos				CRC
		Dirección Comienzo		Número de estados		
addr	01_H	High-Byte	Low-Byte	High-Byte	Low-Byte	crc16

Respuesta : Esclavo → Maestro					
Dirección	Función	Datos			CRC
		Nº Bytes Respuesta	Bytes Respuesta		
addr	01_H	1 byte	1 byte ...	... 1 byte	crc16

Lectura Señales discretas, el Maestro solicita a esclavo 1 leer las salidas discreta, comenzando por la dirección 1 y leyendo 1 punto (Se puede usar 8 salidas; la dirección puede ir de 0 a 7 y los puntos de 1 a 8)

Pregunta : Maestro → Esclavo							
Dirección Esclavo	Función	Datos				CRC	
		Dirección Comienzo		Número de estados			
01_H	01_H	00_H	01_H	00_H	01_H	crc-L	crc-H

- ✓**Dirección Esclavo**:Dirección del dispositivo desde donde se quiere leer.
- ✓**Función**:función requerida = 0x01
- ✓**Dirección Comienzo**:dirección de la salida discreta a partir de donde comienza la lectura.
- ✓**Número de Estados**:Numero de elementos a leer, máximo permitido 0x08.
- ✓**CRC**: Obtenido de la trama a utilizarse para verificar la integridad de la comunicación

- El esclavo responde enviando 1 byte de datos el cual contiene el estado de la salida

Respuesta : Esclavo → Maestro					
Dirección Esclavo	Función	Longitud Datos	Datos	CRC	
01_H	01_H	01_H	01_H	crc-L	crc-H

✓ **Longitud Datos:** Especifica la cantidad de bytes que siguen, sin incluir el CRC

Lee estado entradas digitales. El maestro solicita al esclavo 1 leer las entradas digitales comenzando por la dirección 1 y leyendo 1 punto

Pregunta : Maestro → Esclavo							
Dirección Esclavo	Función	Datos				CRC	
		Dirección Comienzo		Número de estados			
01_H	02_H	00_H	01_H	00_H	04_H	crc-L	crc-H

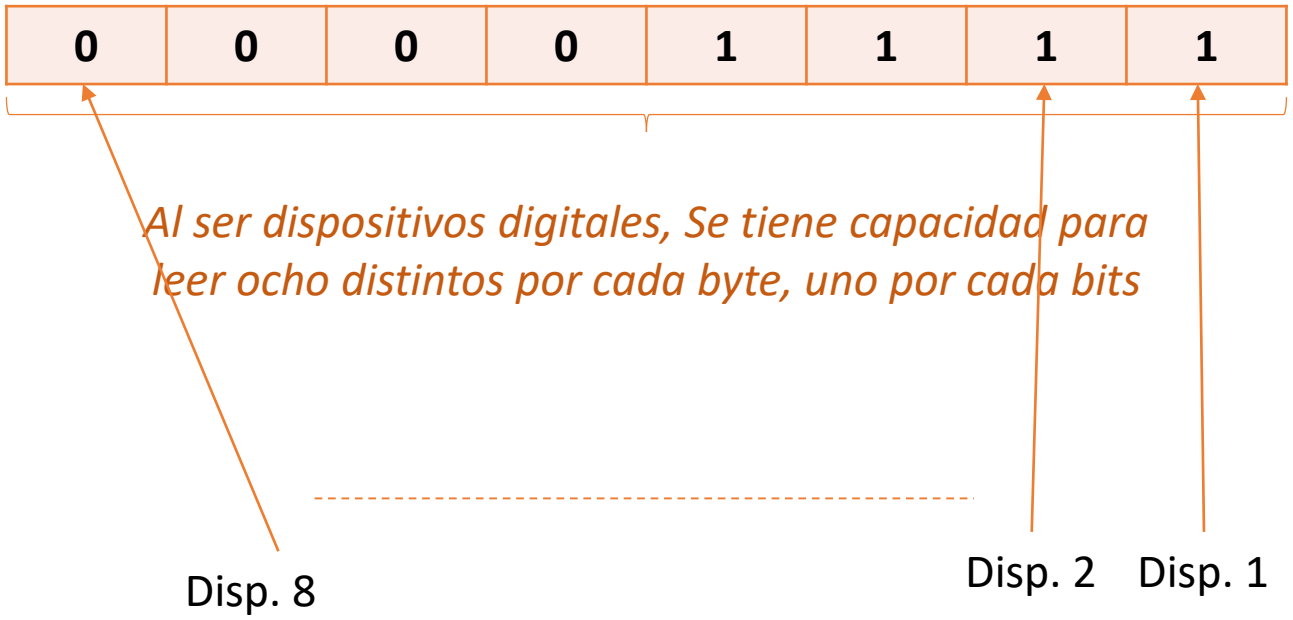
- ✓**Dirección Esclavo:** Dirección del dispositivo desde donde se quiere leer.
- ✓**Función**:función requerida = 0x02
- ✓**Dirección Comienzo:**dirección del registro a partir de donde comienza la lectura. Solo permitido el valor 0x01.
- ✓**Número de Estados:**Numero de elementos a leer.
- ✓**CRC:** Calculado obtenido de la trama a utilizarse para verificar la integridad de la comunicación.

- El esclavo responde enviando 1 byte de datos el cual contiene el estado de la salida

Respuesta : Esclavo → Maestro					
Dirección Esclavo	Función	Longitud Datos	Datos	CRC	
01_H	02_H	01_H	$0F_H$	crc-L	crc-H

✓ **Longitud Datos:** Especifica la cantidad de bytes que siguen, sin incluir el CRC

Significado del byte leído



Leer Registros Analógicos. El maestro solicita al esclavo 1 leer los registros de dirección 0x54 (84) y 0x55 (85)

Pregunta : Maestro → Esclavo							
Dirección Esclavo	Función	Datos				CRC	
		Dirección Comienzo		Número de estados			
01_H	03_H	00_H	54_H	00_H	02_H	crc-L	crc-H

- ✓**Dirección Esclavo:** Dirección del dispositivo desde donde se quiere leer.
- ✓**Función**:función requerida = 0x03
- ✓**Dirección Comienzo:**dirección del registro a partir de donde comienza la lectura.
- ✓**Número de Estados:**Numero de elementos a leer.
- ✓**CRC:** Calculado obtenido de la trama a utilizarse para verificar la integridad de la comunicación.

- El esclavo responde enviando 2 bytes de datos por cada registro solicitado que contiene el valor de los registros solicitados

Respuesta : Esclavo → Maestro								
Dirección Esclavo	Función	Longitud Datos	Datos				CRC	
			H(0x54)	L(0x54)	H(0x55)	L(0x55)		
01_H	03_H	04_H	FF_H	FF_H	FF_H	FF_H	crc-L	crc-H

✓ **Longitud Datos:** Especifica la cantidad de bytes que siguen, sin incluir el CRC